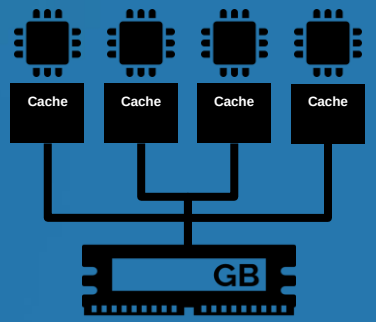
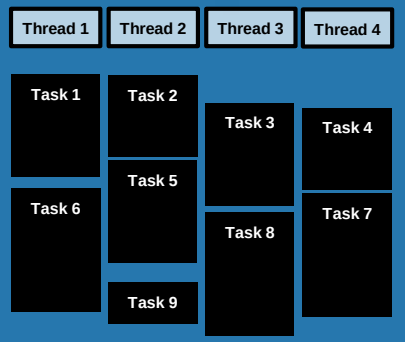
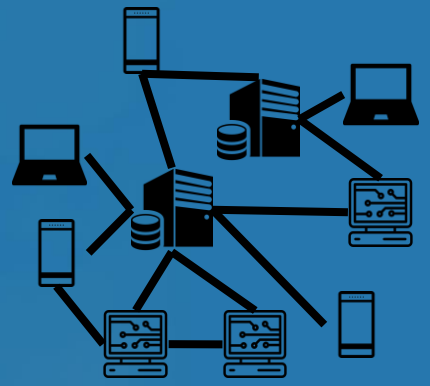
Parameterized Verification 2024







Lecture 4

Model Checking Broadcast Protocols

Let $B = (C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ be the counter system of a parameterized system $sys(P, \mathbb{N}_0)$.

For $R' \subseteq C$, the set of immediate predecessors of R' is

$$pred(R') = \{c \mid \exists c' \in R' \; \exists a \in \Sigma : c \xrightarrow{a} c'\} = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R' \; c \xrightarrow{a} c'\}.$$

I.e., $pred(R')$ is the set of all abstract states that can reach R' in one transition.

Then:

1. if R' is upward-closed, then $pred(R')$ is upward-closed (and both have a finite minimal basis)
2. a finite minimal basis of $pred(R')$ can be computed from a finite minimal basis of R'

Model Checking Broadcast Protocols (IV)

For $R' \subseteq C$, the set of immediate predecessors of R' is

$$pred(R') = \{c \mid \exists c' \in R'\ \exists a \in \Sigma: c \xrightarrow{a} c'\} = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R':\ c \xrightarrow{a} c'\}.$$

Proof idea:

1. Claim: if R' is upward-closed, then $pred(R')$ is upward-closed

$$pred(R') = \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R':\ c \xrightarrow{a} c'\}$$
$$= \bigcup_{a \in \Sigma} \{c \mid \exists c' \in R': c' = M_a \cdot (c - u_i) + u_j \wedge c(i) \geq 1\} \qquad \text{(where } q_i \xrightarrow{a!!} q_j\text{)}$$

Since $M_a \cdot d \preccurlyeq M_a \cdot d'$ and $c + d \preccurlyeq c + d'$ whenever $d \preccurlyeq d'$, each set of the union is upward-closed.

Finally, a finite union of upward-closed sets is again upward-closed.

2. Claim: a finite minimal basis of $pred(R')$ can be computed from a finite minimal basis of R'

One can show that the finite minimal basis of $pred(R')$ are the minimal elements of

$$\bigcup_{a \in \Sigma} \{c \mid \exists c' \in basis(R'): \left[\left(c' = M_a \cdot (c - u_i) + u_j\right) \vee \left(c'(j) = 0 \wedge c' + u_j = M_a \cdot (c - u_i) + u_j\right)\right] \wedge c(i) \geq 1\}$$

case: $c \xrightarrow{a} c'$ case: $c \xrightarrow{a} (c' + u_j)$

Model Checking Broadcast Protocols (V)

This leads us to the following algorithm for checking reachability of $R = \uparrow B$:

1. Let $Initial = \{c \in C \mid c(q) = 0 \text{ if } q \notin Q_0\}$
2. Let $Reached = \uparrow B$ — we start computation with unsafe states
3. Let $Reached' = pred(Reached) \cup Reached$ — we compute backwards with $pred$
4. If $Initial \cap Reached' \neq \emptyset$ then return "reachable" — check if initial states have been reached
5. If $Reached' \neq Reached$ then {let $Reached = Reached'$; goto 3} — check if any new states have been reached; if so, repeat
6. Return "unreachable" — if we reached a fixpoint, but no initial states, then $\uparrow R$ is unreachable

$Reached$ is always upward-closed and can be represented by a finite minimal basis

Decidability Results: Reachability Properties in (Pairwise and Asynchronous) Rendezvous Systems

Well-Structured Transition Systems (II)

For a (possibly infinite-state) transition system (S, S_0, Σ, Δ), we write $s \to^* s'$ if there exists a path from s to s'.

We say that a wqo $\lesssim \subseteq S \times S$ is compatible with Δ if the following holds:

whenever $s \to s'$ and $s \lesssim t$ then there exists t' with $s' \lesssim t'$ and $t \to^* t'$.

$$\begin{array}{ccc} t & \to^* & t' \\ \wr\vee & & \wr\vee \\ s & \to & s' \end{array}$$

We say $\lesssim$ is strongly compatible with Δ if the above holds with $t \to t'$ instead of $t \to^* t'$.

Well-Structured Transition Systems (III)

An infinite-state labeled transition system (S, S_0, Σ, Δ) is a well-structured transition system (WSTS) if there exists a wqo $\lesssim \subseteq S \times S$ that is compatible with Δ.

Let $R \subseteq S$ and let $pred(R)$ be the set of immediate predecessors of R (with respect to Δ).

If R is upward-closed (wrt. $\lesssim$), then for the sequence $R_0 \subseteq R_1 \subseteq \cdots$ with $R_0 = R$ and $R_{i+1} = R_i \cup pred(R_i)$ there exists $k \in \mathbb{N}$ with $R_{k+1} = R_k$. We write $pred^*(R)$ for this R_k.

Note: In this sequence, every R_i is upward-closed. Therefore, $pred^*(R)$ is upward-closed.

We say a WSTS has effective pred-basis if there exists an algorithm that takes as input any state $s \in S$ and returns a finite basis of $\uparrow pred(\uparrow s)$.

Observation: For a WSTS with effective pred-basis, we can compute a finite basis of $pred(R)$ for any upward-closed R that is given as a finite basis.

Well-Structured Transition Systems (IV)

We call a set $I \subseteq S$ downward-closed if whenever $s \in I$ and $s' \lesssim s$, then $s' \in I$.

Example: in the counter system of a parameterized system, the set of initial states is always downward-closed.

Note: downward-closed sets do not necessarily have a finite basis.

Theorem:

For a WSTS with effective pred-basis, we can decide reachability of an upward-closed set R from an initial set of states I if either

- I is upward-closed and we can decide emptiness of $U \cap U'$ for any upward-closed sets U, U'

or

- I is downward-closed and we can decide emptiness of $D \cap U$ for any downward-closed set D and upward-closed set U

Counter Representation of other Parameterized Systems

For systems with pairwise rendezvous ($card = \{1\}$) or asynchronous rendezvous ($card = \{0,1\}$), the counter system $(C, C_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta_{abs})$ of $sys(P, card)$ is defined in the same way as for broadcast protocols.

Computing the effect of a transition is even simpler:

- a pairwise rendezvous transition $c \xrightarrow{a} c'$ based on $q_i \xrightarrow{a!} q_j$ and $q_k \xrightarrow{a?} q_l$ can be computed by letting $c' = c - u_i + u_j - u_k + u_l$, with the condition that $c(q_i) \geq 1$ and $c(q_k) \geq 1$
- an asynchronous rendezvous transition $c \xrightarrow{a} c'$ is either computed in the same way, or by letting $c' = c - u_i + u_j$, with the condition that $c(q_i) \geq 1$ and $\nexists\, q_k$ with $q_k \xrightarrow{a?} q_l$ and $c(q_k) \geq 1$

Parameterized Systems as WSTSs

Theorem:
If $P = (Q, Q_0, \Sigma, \delta, \lambda)$ is a process based on actions Σ_{int} and Σ_{sync}, and $card \subseteq \mathbb{N}_0$ is an interval, then the counter system of $sys(P, card)$ with the wqo $\preccurlyeq$ is a WSTS with effective pred-basis.

Corollary:
The counter system of a system with pairwise rendezvous or asynchronous rendezvous synchronization is a WSTS with effective pred-basis wrt. $\preccurlyeq$.

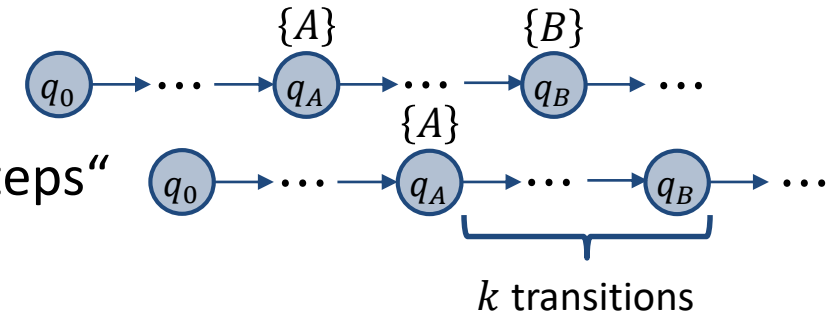
Decidability Results:
Beyond Reachability Properties

Safety and Liveness Properties (II)

A property of the (parameterized) system is a property that holds for all runs of the system.

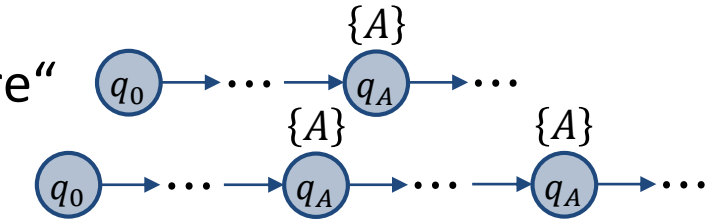
Safety properties are properties that are violated after finite time, even in infinite runs:

- „if A happens, then B will not happen afterwards“
- „whenever A happens, then B will happen within k steps“



Liveness properties can only be violated in inifinite time

- „whenever A happens, then B will happen at some point in the future“
- „at every point of the execution, A will happen again in the future“



For finite state transition systems, model checking is decidable for both safety and liveness properties.

For many parameterized systems, the difference between safety and liveness determines whether the PMCP is decidable or not.

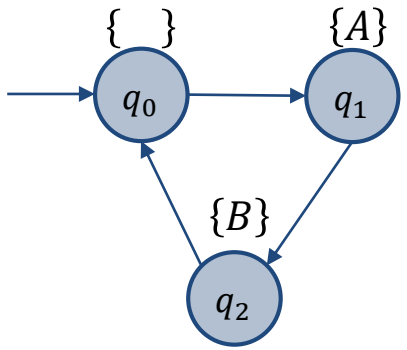
Checking Safety Properties: Finite-State Automata (II)

To check whether a safety property is satisfied by an LTS M, we construct the composition of M with the automaton that accepts all words that violate the property.

The composition $M \times A$ of an LTS $M = (Q, Q_0, \Sigma, \delta, \lambda)$ with a finite-state automaton $A = (Q_A, q_{A,0}, \Sigma_A, \delta_A)$, where $\lambda: Q \to 2^{AP}$ and $\Sigma_A = 2^{AP}$, is the transition system $(Q \times Q_A, Q_0 \times \{q_{A,0}\}, \Sigma, \Delta)$ with $((q, q_A), a, (q', q'_A)) \in \Delta$ if $(q, a, q') \in \delta$ and $(q_A, \lambda(q), q'_A) \in \delta_A$

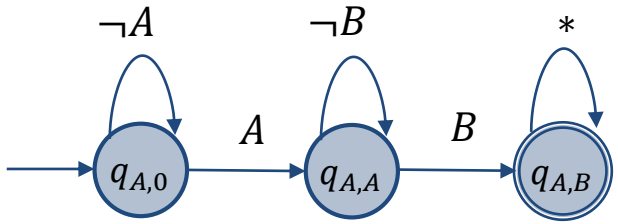
Example:

M

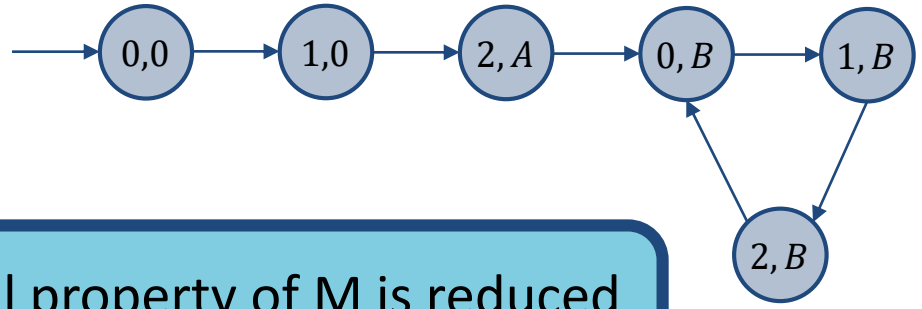


(omitting actions)

A



$M \times A$



Checking the original property of M is reduced to checking state reachability in $M \times A$

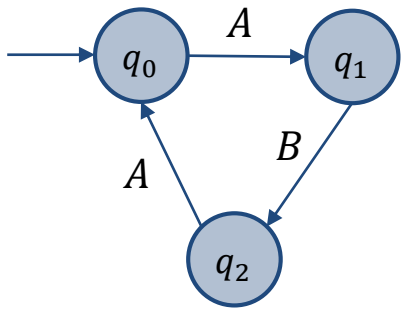
Checking Safety Properties: Finite-State Automata (III)

Instead of the state labels, the automaton can also read the actions of an LTS. This allows us to use the automaton to check properties of the actions along runs of the system.

The composition $M \times A$ of an LTS $M = (Q, Q_0, \Sigma, \delta, \lambda)$ with a finite-state automaton $A = (Q_A, q_{A,0}, \Sigma_A, \delta_A)$, where $\Sigma_A = \Sigma$, is the transition system $(Q \times Q_A, Q_0 \times \{q_{A,0}\}, \Sigma, \Delta)$ with $((q, q_A), a, (q', q'_A)) \in \Delta$ if $(q, a, q') \in \delta$ and $(q_A, a, q'_A) \in \delta_A$

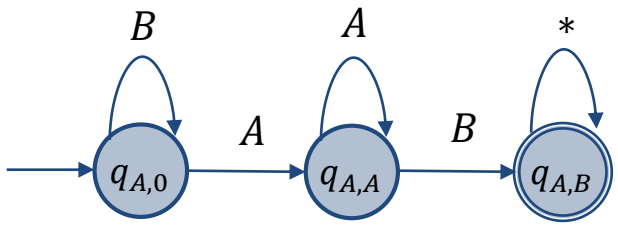
Example:

M

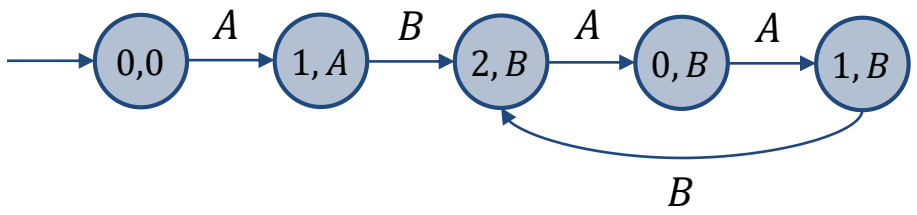


(omitting state labels)

A



$M \times A$



Checking Arbitrary Safety Properties in Parameterized Systems

Even for the case of specifications on actions, where $M \times A$ can be constructed directly, it is not immediately clear if the PMCP is still decidable, since we don't know that the resulting composed system is still a WSTS.

Theorem:

Let $M = (S, S_0, \Sigma, \Delta)$ be a WSTS with effective pred-basis wrt. $\lesssim$, and $A = (Q, q_0, \Sigma, \delta)$ a finite-state automaton. Then $M \times A$ is a WSTS with effective pred-basis wrt. the wqo $\lesssim_A$ defined by

$$(s, q) \lesssim_A (s', q') \quad \Leftrightarrow \quad s \lesssim s' \wedge q = q'$$

Preview and Learning Goals for this Lecture

Topics:

- Example: Checking a Complex Safety Property of the MESI Protocol
- (A Closer Look at) Systems with Pairwise Rendezvous (PR) Synchronization
- Checking Liveness Properties of PR Systems
- Liveness and Fairness

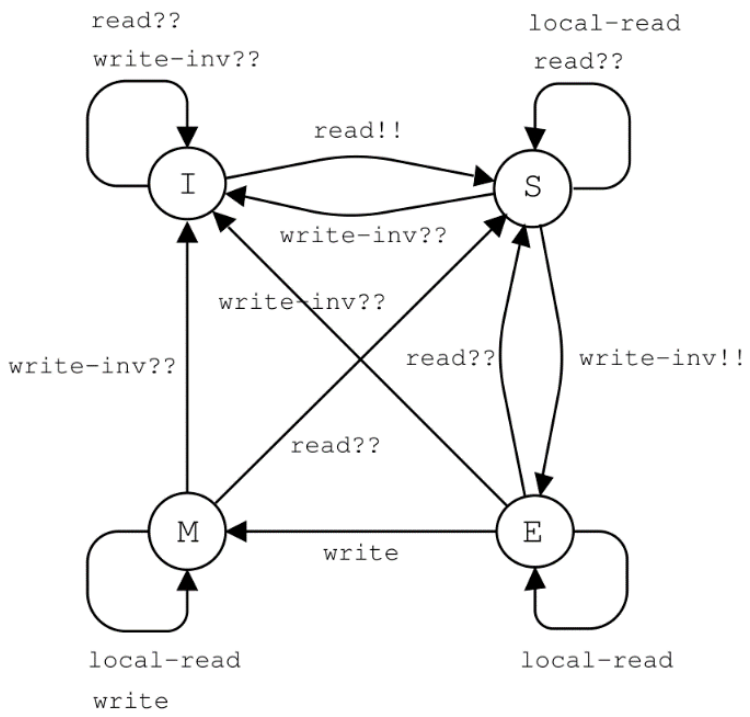
Techniques:

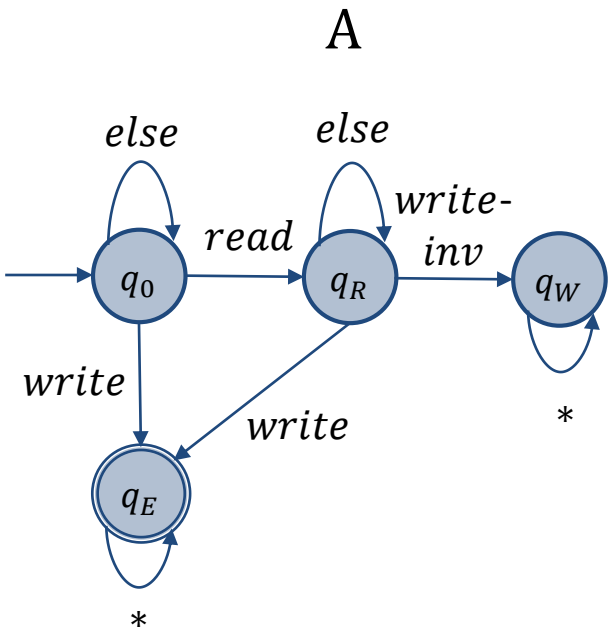
- Vector Addition Systems with States (VASS)
- Modifications of VASS to check Safety and Liveness Properties

Example: Checking a Complex Safety Property of the MESI Protocol

We can use the approach to check properties of the sequences of actions of a broadcast protocol.

To this end, construct the combination of the counter system M and the automaton A that detects violation of the specification. The result is a WSTS wrt. $\preccurlyeq_A$, the modified version of $\preccurlyeq$.





"Before the first write, there must be a read and a write-inv."

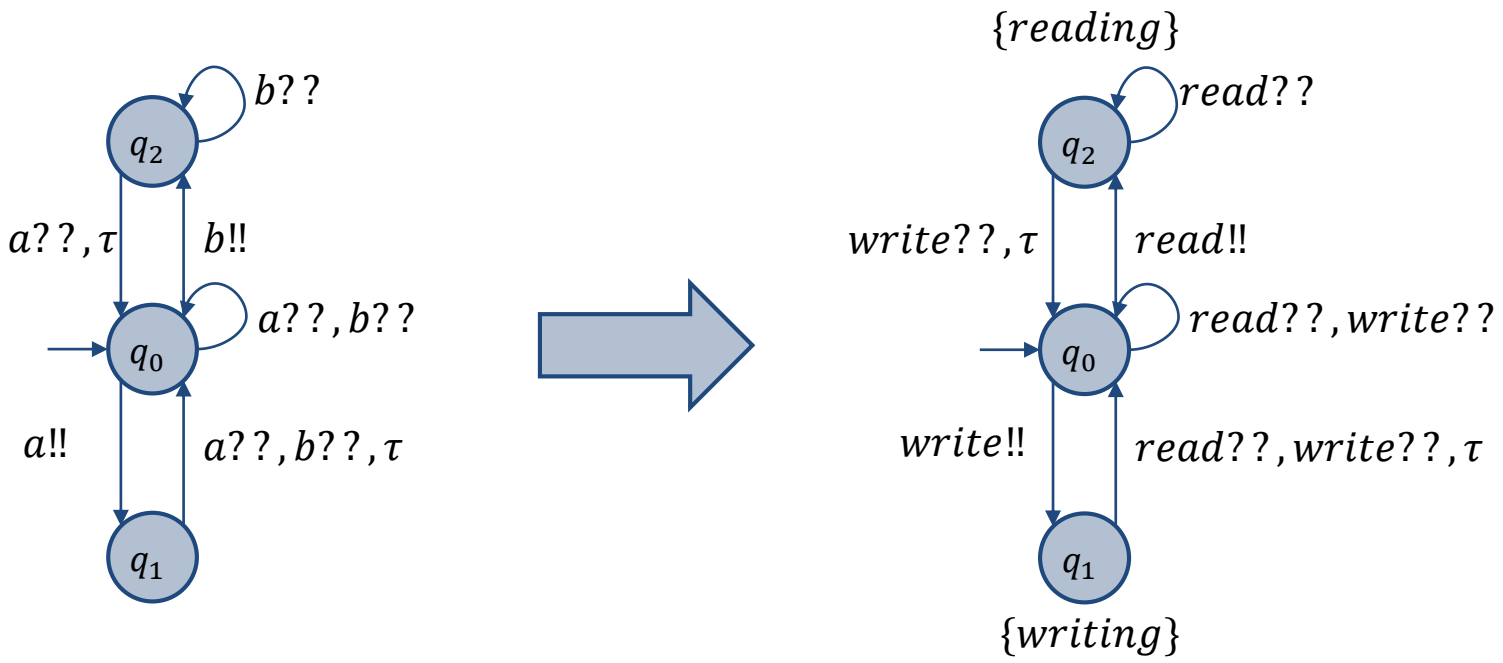
Example: Checking a Complex Safety Property of the MESI Protocol

Based on $M \times A$, parameterized model checking now works as before, except that

- $\uparrow R = \uparrow \{((0,0,0,0), q_E)\}$
- $pred(Reached)$ is now computed on $S \times Q$
  (which can be done independently in the two components, we only need to ensure that both transitions use the same action)
- the initial states are $\downarrow ((\infty, 0,0,0), q_0)$

Modeling Systems with Pairwise Rendezvous

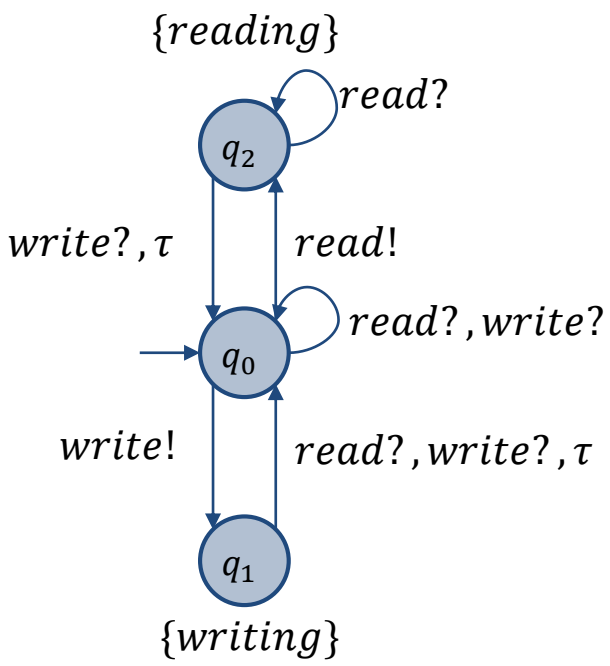
Review: Simple Reader-Writer-Protocol in Broadcast Protocol



Safety properties of interest:

- never two (or more) processes in a $writing$ state at the same time
- never a process in a $reading$ state if there is also a process in a $writing$ state

Example: Simple Reader-Writer in PR System?



Safety properties of interest:

- never two (or more) processes in a $writing$ state at the same time
- never a process in a $reading$ state if there is also a process in a $writing$ state

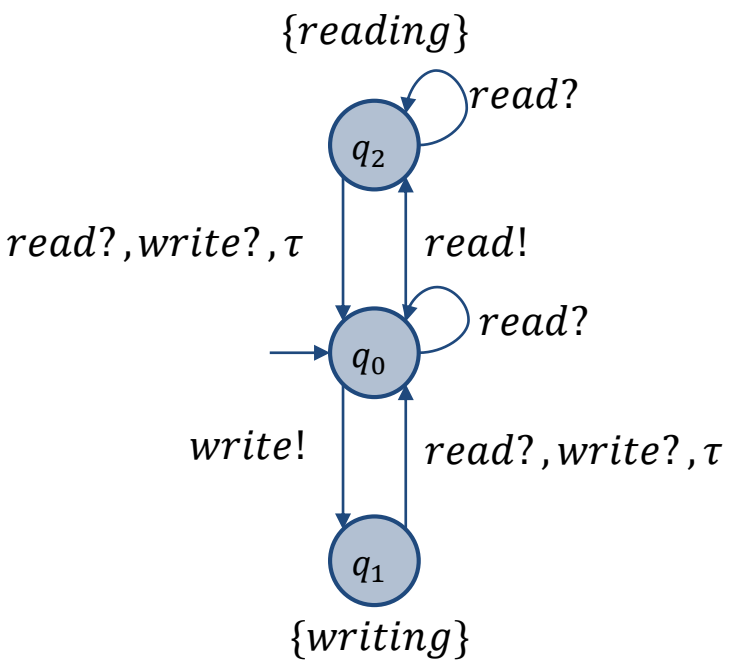
First try: replace broadcast actions with PR actions

Satisfies both properties for system with two processes

But not with three or more:

if there is a process p in q_1 and another process p' wants to move from q_0 to q_1, then p' can synchronize with p, but it can also synchronize with any other process in the system, which results in both p and p' being in q_1 at the same time

Example: Simple Reader-Writer in PR System?



Safety properties of interest:

- never two (or more) processes in a $writing$ state at the same time
- never a process in a $reading$ state if there is also a process in a $writing$ state

Workaround: remove $write?$ action from self-loop on q_0

Then we could still move two processes into q_2, and then two processes into q_1

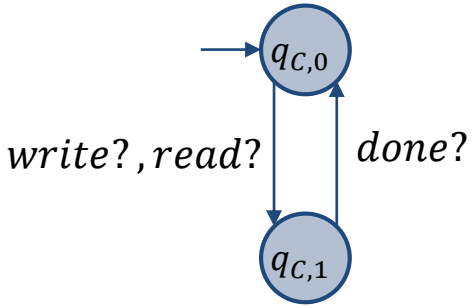
Second workaround: also remove self-loop with $read?$ on q_0 and q_2, and add transition $q_2 \xrightarrow{read?} q_0$ with

Now both properties will hold, but we have a deadlock in the initial state

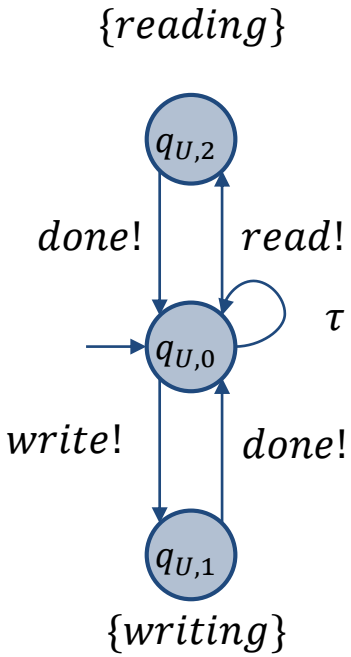
Simple Reader-Writer in PR System with Controller Process

Suppose we have a system with

a single controller process C:



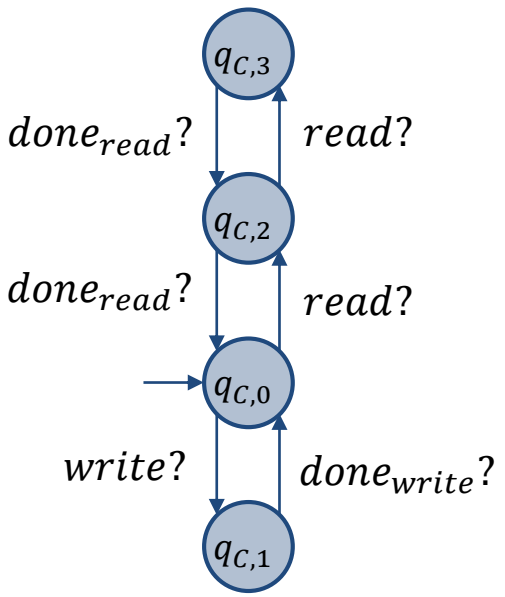
arbitrarily many user processes U:

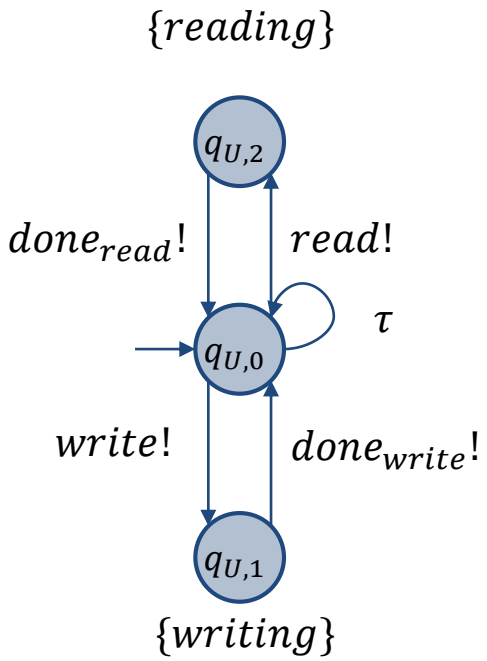


Both properties are satisfied, but: only one process at a time can be in $reading$ state

Simple Reader-Writer in PR System with Controller Process (II)

We can work around this by separating $done$ actions into $done_{read}$ and $done_{write}$, and adding states to the controller process, e.g. to allow two processes to read at the same time:





This can already be modeled (and checked) in the WSTS framework

This allows multiple processes to read at the same time, but only for a fixed number.

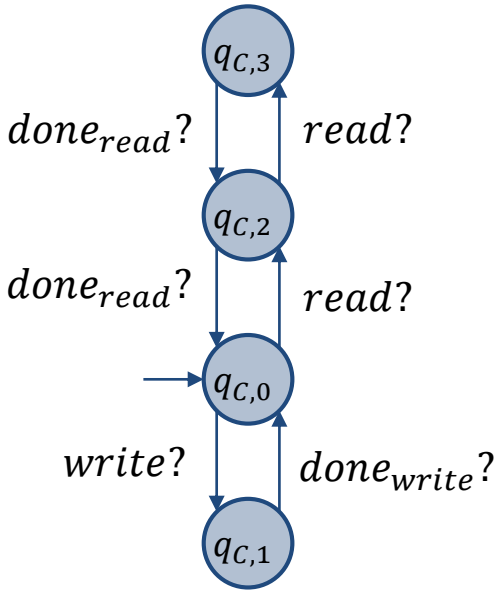
Also: a $write$ transition can only happen if controller returns to state q_0

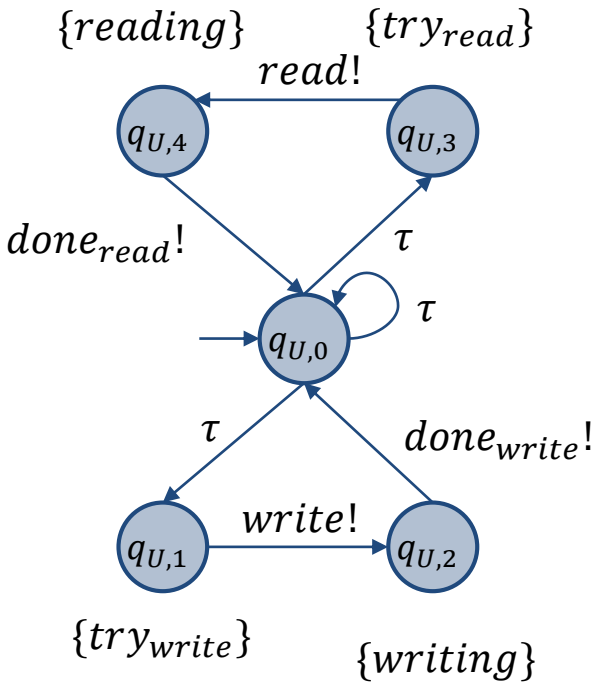
Advanced Reader-Writer in PR System with Controller Process

Suppose we want to guarantee an additional liveness property

"every process that wants to write will eventually be in a writing state",

and similarly for reading. To model this, add states that signal "wants to write" and "wants to read":





Checking liveness properties is not supported by the WSTS framework

Then we can express our properties as

"every process that enters a try_{write} state will eventually enter a $writing$ state",

and similarly for $reading$

Vector Addition Systems with States

Motivation

We want to

- model PR systems with controller process, and
- reason about safety and liveness properties.

To this end, we introduce a model for parameterized systems that can be seen as a special form of the counter representation we used for broadcast protocols.

We will

1. introduce the new model,
2. show how it can represent PR systems, and
3. show how we can reason about safety and liveness properties in this model.

Vector Addition System with States

A vector addition system with states (VASS) is a tuple (Q, q_0, A, T), where

- Q is a finite set of states
- $q_0 \in Q$ is an initial state
- $A \subset \mathbb{Z}^m$ is a finite set of actions
- $T \subseteq Q \times A \times Q$ is a transition relation

We call a tuple $(q, \mathbf{v}) \in Q \times \mathbb{N}_0^m$ a configuration of the VASS, and $\mathbf{v} \in \mathbb{N}_0^m$ the configuration vector.

Then the configuration space of a VASS (Q, q_0, A, T) is the (possibly infinite-state) transition system $(Q \times \mathbb{N}_0^m, (q_0, \vec{0}), A, \Delta)$, where

Δ contains a transition $(q, \mathbf{v}) \xrightarrow{\mathbf{a}} (q', \mathbf{v}')$ if $(q, \mathbf{a}, q') \in T$ and $\mathbf{v}' = \mathbf{v} + \mathbf{a}$ with $\mathbf{v}' \in \mathbb{N}_0^m$

Representing PR Systems as VASSs

Consider the PR system with controller process $C = (Q_C, q_{C,0}, \Sigma, \delta_C, \lambda_C)$ and user processes $U = (Q_U, q_{U,0}, \Sigma, \delta_U, \lambda_U)$, and let $m = |Q_U|$.

Let $\mathbf{u}_i \in \mathbb{Z}^m$ be the unit vector with $\mathbf{u}_i(i) = 1$, let $\mathbf{v}_{i,j} = -\mathbf{u}_i + \mathbf{u}_j$ and $\mathbf{v}_{i,j,k,l} = -u_i + u_j - u_k + u_l$.

Idea: use a VASS to represent the parameterized PR system, similar to the counter representation for broadcast protocols.

The PR system can be represented as the VASS with $A \subset \mathbb{Z}^m$, $Q = Q_C \cup \{q_{init}\}$, $q_0 = q_{init}$, and

$$A = \{\vec{0}\} \cup \{\mathbf{u}_0\} \cup \{\mathbf{v}_{i,j}\}_{q_i, q_j \in Q_U} \cup \{\mathbf{v}_{i,j,k,l}\}_{q_i, q_j, q_k, q_l \in Q_U}$$

with T defined on the next slide.

Representing PR Systems as VASSs (II)

(Reminder: $\mathbf{v}_{i,j} = -\mathbf{u}_i + \mathbf{u}_j$ and $\mathbf{v}_{i,j,k,l} = -\mathbf{u}_i + \mathbf{u}_j - \mathbf{u}_k + \mathbf{u}_l$)

$$T = \begin{cases} & \{(q_{init}, \mathbf{u}_0, q_{init})\} \\ \cup & \{(q_{init}, \vec{0}, q_{C,0})\} \\ \cup & \{(q, \mathbf{v}_{i,j}, q) \mid q_i \xrightarrow{\tau} q_j \text{ in } U\} \\ \cup & \left\{(q, \mathbf{v}_{i,j,k,l}, q) \;\middle|\; q_i \xrightarrow{a!} q_j \text{ and } q_k \xrightarrow{a?} q_l \text{ in } U\right\}^* \\ \cup & \left\{(q, \vec{0}, q') \;\middle|\; q \xrightarrow{\tau} q' \text{in } C\right\} \\ \cup & \left\{(q, \mathbf{v}_{i,j}, q') \;\middle|\; q_i \xrightarrow{a!} q_j \text{ in } U \text{ and } q \xrightarrow{a?} q' \text{in } C\right\}^* \\ \cup & \left\{(q, \mathbf{v}_{i,j}, q') \;\middle|\; q_i \xrightarrow{a?} q_j \text{ in } U \text{ and } q \xrightarrow{a!} q' \text{in } C\right\}^* \end{cases}$$

Note: configuration space of the VASS has initial configuration defined as $(q_0, \vec{0})$.

To simulate a PR system, use new initial state q_{init}, and transitions $(q_{init}, \mathbf{u}_0, q_{init})$ and $(q_{init}, \vec{0}, q_{C,0})$ that first "initialize" the system to an arbitrary size, and then start execution of the system.

*: to avoid simulating transitions that are impossible, we need to add intermediate states to the VASS if $i = l, j = k, i = j$ or $k = l$, and simulate the local transitions separately. (since the vector will evaluate to 0 in that position, not reflecting that we need a process in this local state)

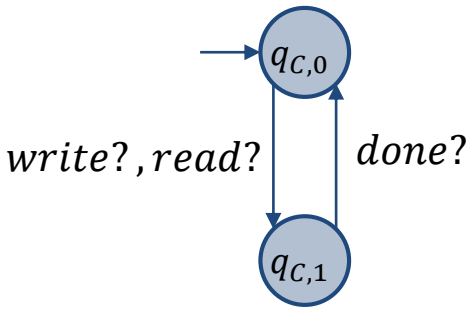
Example: Simple Reader-Writer with Controller as a VASS

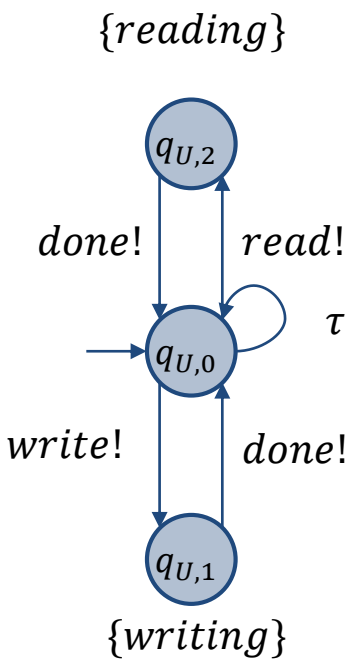
$$Q = \{q_{init}, q_{C,0}, q_{C,1}\}$$

$$q_0 = q_{init}$$

$$A = \{(0,0,0), (+1,0,0), (-1,+1,0), (-1,0,+1), (+1,-1,0), (+1,0,-1)\}$$

$$T = \left\{\begin{array}{c} (q_{init}, (+1,0,0), q_{init}) \\ (q_{init}, (0,0,0), q_{C,0}) \\ (q_{C,0}, (0,0,0), q_{C,0}) \\ (q_{C,0}, (-1,+1,0), q_{C,1}), \\ (q_{C,0}, (-1,0,+1), q_{C,1}), \\ (q_{C,1}, (0,0,0), q_{C,1}), \\ (q_{C,1}, (+1,-1,0), q_{C,0}), \\ (q_{C,1}, (+1,0,-1), q_{C,0}) \end{array}\right\}$$





Initial configuration: $(q_{init}, \vec{0})$

Representing PR Systems as VASSs (III)

Note: the VASS representation

- is more general than the counter representation in that it includes a controller process explicitly; later we will see that this extends to any fixed number of processes
  - ➢ this allows us to reason about state labels of the processes that are modeled explicitly
- is more restricted than the counter representation in that it only allows transitions where updates are by addition of a vector (and update of the explicitly modeled processes)
  - ➢ this does not allow us to model broadcast protocols, but the restriction is necessary to obtain decidability for liveness properties

The generalization of the counter representation can also be used without the restriction, allowing us to check arbitrary safety properties (on the state labels) of any fixed number of processes for broadcast protocols and other WSTS, based on what we will see in the following.

Checking Reachability in a VASS

Observation:

The configuration space of any VASS with wqo

$$(q, \mathbf{v}) \lesssim (q, \mathbf{v}') \quad \Leftrightarrow \quad q = q' \wedge \bigwedge_{1 \leq i \leq n} \mathbf{v}(i) \lesssim \mathbf{v}'(i)$$

is a WSTS (with a single initial state).

Therefore:

Checking reachability of upward-closed sets of states is decidable for VASSs.
(We can use an adapted version of the model checking algorithm for broadcast protocols.)

Checking Safety Properties of PR Systems

Consider a PR system with controller process $C = (Q_C, q_{C,0}, \Sigma, \delta_C, \lambda_C)$, where $\lambda_C : Q_C \to 2^{AP}$, and user processes $U = (Q_U, q_{U,0}, \Sigma, \delta_U, \lambda_U)$.

To check safety properties on the state labels of the controller process:

1. let $AP' = AP \cup \{init\}$ for a fresh atomic proposition $init$, and extend λ_C to $Q_C \cup \{q_{init}\}$ by letting $\lambda'_C(q_{init}) = \{init\}$ and $\lambda'_C(q_C) = \lambda_C(q_C)$ for $q_C \in Q_C$
2. as seen before, encode desired property into an automaton $A = (Q_A, q_{A,0}, \Sigma_A, \delta_A, F_A)$ with $\Sigma_A = 2^{AP'}$ that reads state labels of controller and accepts all paths that violate the property. For this automaton, let $(q_{A,0}, \{init\}, q_{A,0})$ be the only transition on $\{init\}$ in δ_A
3. instead of the VASS $(\{q_{init}\} \cup Q_C, q_{init}, A, T)$, consider the VASS
   $\left((\{q_{init}\} \cup Q_C) \times Q_A, (q_{init}, q_{A,0}), A, \Delta\right)$
   where $((q_C, q_A), \mathbf{a}, (q'_C, q'_A)) \in \Delta$ if $(q_C, \mathbf{a}, q'_C) \in T$ and $(q_A, \lambda'_C(q_C), q'_A) \in \delta_A$
4. check for reachability of a configuration $((q_C, q_A), \mathbf{v})$ where $q_A \in F_A$

Checking Safety Properties of PR Systems (II)

Consider a PR system with controller process $C = (Q_C, q_{C,0}, \Sigma, \delta_C, \lambda_C)$ and user processes $U = (Q_U, q_{U,0}, \Sigma, \delta_U, \lambda_U)$, where $\lambda_U : Q_U \to 2^{AP}$.

To check safety properties on the state labels of a single user process:

1. let $AP' = AP \cup \{init\}$ for a fresh atomic proposition $init$, and extend λ_U to $Q_U \cup \{q_{init}\}$ by letting $\lambda'_U(q_{init}) = \{init\}$ and $\lambda'_U(q_U) = \lambda_U(q_U)$ for $q_U \in Q_U$
2. as seen before, encode desired property into an automaton $A = (Q_A, q_{A,0}, \Sigma_A, \delta_A, F_A)$ with $\Sigma_A = 2^{AP'}$ that reads state labels of user process and accepts all paths that violate the property. For this automaton, let $(q_{A,0}, \{init\}, q_{A,0})$ be the only transition on $\{init\}$ in δ_A
3. instead of the VASS $(\{q_{init}\} \cup Q_C, q_{init}, A, T)$, consider the VASS $(Q_C \times (\{q_{init}\} \cup Q_U) \times Q_A, (q_{C,0}, q_{init}, q_{A,0}), A, \Delta)$ with Δ defined on next slide
4. check for reachability of a configuration $((q_C, q_U, q_A), \mathbf{v})$ where $q_A \in F_A$

Checking Safety Properties of PR Systems (III)

Idea: for every transition that involves some user process, need to add a transition of the user process that is now modeled explicitly

Transition relation of VASS $\left(Q_C \times (\{q_{init}\} \cup Q_U) \times Q_A, (q_{C,0}, q_{init}, q_{A,0}), A, \Delta\right)$:

$$\Delta = \begin{cases} \left\{\left((q_{C,0}, q_{init}, q_{A,0}), \mathbf{u}_0, (q_{C,0}, q_{init}, q_{A,0})\right), \left((q_{C,0}, q_{init}, q_{A,0}), \vec{0}, (q_{C,0}, q_{U,0}, q_{A,0})\right)\right\} \\ \cup \ \left\{\left((q_C, q_U, q_A), \mathbf{a}, (q'_C, q_U, q'_A)\right) \mid (q_C, \mathbf{a}, q'_C) \in T, q_C \neq q_{init} \text{ and } (q_A, \lambda'_U(q_U), q'_A) \in \delta_A\right\} \\ \cup \ \left\{\left((q_C, q_U, q_A), \vec{0}, (q_C, q'_U, q'_A)\right) \mid q_U \xrightarrow{\tau} q'_U \text{ in } U \text{ and } (q_A, \lambda'_U(q_U), q'_A) \in \delta_A\right\} \\ \cup \ \left\{\left((q_C, q_U, q_A), \mathbf{v}_{i,j}, (q_C, q'_U, q'_A)\right) \mid q_U \xrightarrow{a!} q'_U \text{ and } q_i \xrightarrow{a?} q_j \text{ in } U \text{ and } (q_A, \lambda'_U(q_U), q'_A) \in \delta_A\right\} \\ \cup \ \left\{\left((q_C, q_U, q_A), \mathbf{v}_{i,j}, (q_C, q'_U, q'_A)\right) \mid q_U \xrightarrow{a?} q'_U \text{ and } q_i \xrightarrow{a!} q_j \text{ in } U \text{ and } (q_A, \lambda'_U(q_U), q'_A) \in \delta_A\right\} \\ \cup \ \left\{\left((q_C, q_U, q_A), \vec{0}, (q'_C, q'_U, q'_A)\right) \mid q_U \xrightarrow{a!} q'_U \text{ in } U, q_C \xrightarrow{a?} q'_C \text{ in } C \text{ and } (q_A, \lambda'_U(q_U), q'_A) \in \delta_A\right\} \\ \cup \ \left\{\left((q_C, q_U, q_A), \vec{0}, (q'_C, q'_U, q'_A)\right) \mid q_U \xrightarrow{a?} q'_U \text{ in } U, q_C \xrightarrow{a!} q'_C \text{ in } C \text{ and } (q_A, \lambda'_U(q_U), q'_A) \in \delta_A\right\} \end{cases}$$

Checking Safety Properties of PR Systems (IV)

Why did we change the construction such that U starts in q_{init} instead of C?

Because otherwise the automaton would read the state label of $q_{U,0}$ in every step of the initialization phase. We want to start it reading state labels (other than $init$) only when initialization is over.

In a similar way as seen on the last slides, we can check safety properties of the controller process and any fixed number of user processes by

- defining an autmaton that reads the labels of multiple processes
- including all of these processes explicitly in the state space Q of the VASS (such that they stay in a special initialization state until the initialization is over)

Checking Liveness Properties of PR Systems

Checking Liveness Properties: Büchi Automata

To check liveness properties, we also encode them as automata.

A Büchi automaton is a tuple $(Q, q_0, \Sigma, \delta, F)$ where

- Q is a finite set of states
- $q_0 \in Q$ is the initial state
- Σ is an input alphabet
- $\delta \subseteq Q \times \Sigma \times Q$ is a transition relation
- $F \subseteq Q$ is a set of accepting states

Note that all components are the same as for finite-state automata.

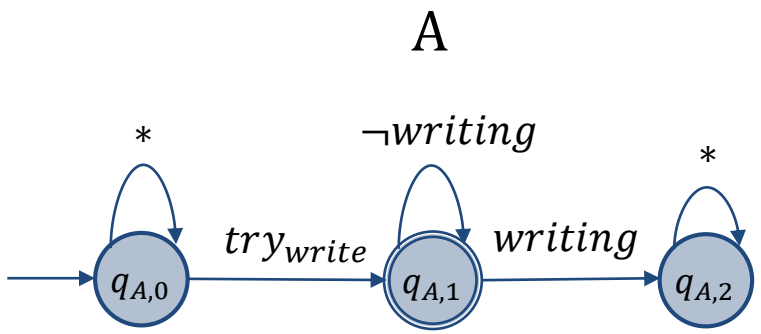
A run of the automaton is an infinite sequence $q_1 a_1 q_2 a_2 \ldots$ in $(Q\Sigma)^\omega$ with $q_1 = q_0$ and $(q_i, a_i q_{i+1}) \in \delta$ for $i \geq 1$. We say it is a run on the ω-word $w = a_1 a_2 \ldots \in \Sigma^\omega$.

The run is accepting if there is $q \in F$ that appears infinitely often in the run. An ω-word $w \in \Sigma^\omega$ is accepted by the automaton if there exists an accepting run of the automaton on w.

Example: Liveness Property as Büchi Automaton

We want an automaton that accepts all runs that violate the property

"if the user process enters a try_{write} state then it will eventually enter a $writing$ state"



Note: automaton A is non-deterministic – it can stay in $q_{A,0}$ for an arbitrary time.
Upon reading try_{write}, it can either move into $q_{A,1}$ or stay in $q_{A,0}$.
It accepts only if after moving into $q_{A,1}$ it never reads $writing$ again.
Thus, an ω-word is accepted iff there is some try_{write} that is not followed by a $writing$.

Checking Liveness Properties of PR Systems

Let $C \parallel U^n$ be the PR system with controller process C and n user processes U

Let f be a liveness property of C and $A_{\neg f}$ an automaton that accepts all runs that do not satisfy f

Let $VASS(C, U^i, A_{\neg f})$ be the configuration space of the VASS constructed from C, U and $A_{\neg f}$, with i explicit copies of U. We call a configuration of this VASS an accepting configuration if it contains an accepting state of $A_{\neg f}$

Lemma: There is an $n \in \mathbb{N}$ and a run of $C \parallel U^n$ that does not satisfy f
iff $VASS(C, U^0, A_{\neg f})$ has an infinite run that contains infinitely many accepting configurations.

Proof idea:
$\Rightarrow$: follows from the ability of the VASS to „load“ a system of size n and correctly simulate the run

$\Leftarrow$: follows from the fact that the VASS has to „load“ a system of some size n and then correctly simulates a run (and from the properties of $A_{\neg f}$)

Checking Liveness Properties of PR Systems (II)

Let f be a liveness property of U and $A_{\neg f}$ an automaton that accepts all runs that do not satisfy f

Lemma: There is an $n \in \mathbb{N}$ and a run of $C \parallel U^n$ that does not satisfy f
iff $VASS(C, U^1, A_{\neg f})$ has an infinite run that contains infinitely many accepting configurations.

Note: if a liveness (or safety) property holds for one of the user processes, then by symmetry it holds for all user processes, since all of them start in the same initial state and every run that is possible for one user process is possible for all user processes.

Checking Liveness Properties of PR Systems (III)

Lemma: $VASS(C, U^i, A)$ has an infinite run with infinitely many accepting configurations iff it has a finite path of the form $\alpha\beta$ with the following properties:

1. α starts in the initial configuration
2. β is a cycle, i.e., it starts and ends in the same configuration
3. except for the first configuration of β, no configuration appears more than once on $\alpha\beta$
4. a final configuration appears in β

Proof idea: follows from the fact that there are only finitely many accepting states of A, and, even though $VASS(C, U^i, A)$ is an infinite-state system, after the initialization it simulates a system of fixed size n.
Therefore, any infinite run must contain repetitions, and any run that visits infinitely many accepting configurations can be partitioned into the form $\alpha\beta^\omega$ with the properties above.

Checking Liveness Properties of PR Systems (IV)

Let $p = \left|(Q_C \cup \{q_{init}\}) \times (Q_U)^i \times Q_A\right|$ and $m = |Q_U|$.

Lemma: There exists a finite path of $VASS\left(C, U^i, A\right)$ of the form $\alpha\beta$ with the properties from the preceding Lemma iff there exists such a path of length at most $2^{k \cdot p \cdot \log(p) \cdot 2^{k \cdot m \cdot \log(m)}}$, for some constant k.

(length of the path follows from how many configurations we can see without a repetition)

Theorem: The PMCP for PR systems with finite-state processes and properties defined as Büchi automata is decidable (in time exponential in $|Q_C|$ and $|A|$ and doubly exponential in $|Q_U|$).

Corollary: There exists a cutoff for the PMCP of PR systems and properties defined by a Büchi automaton A, where U, C and A are of bounded size.

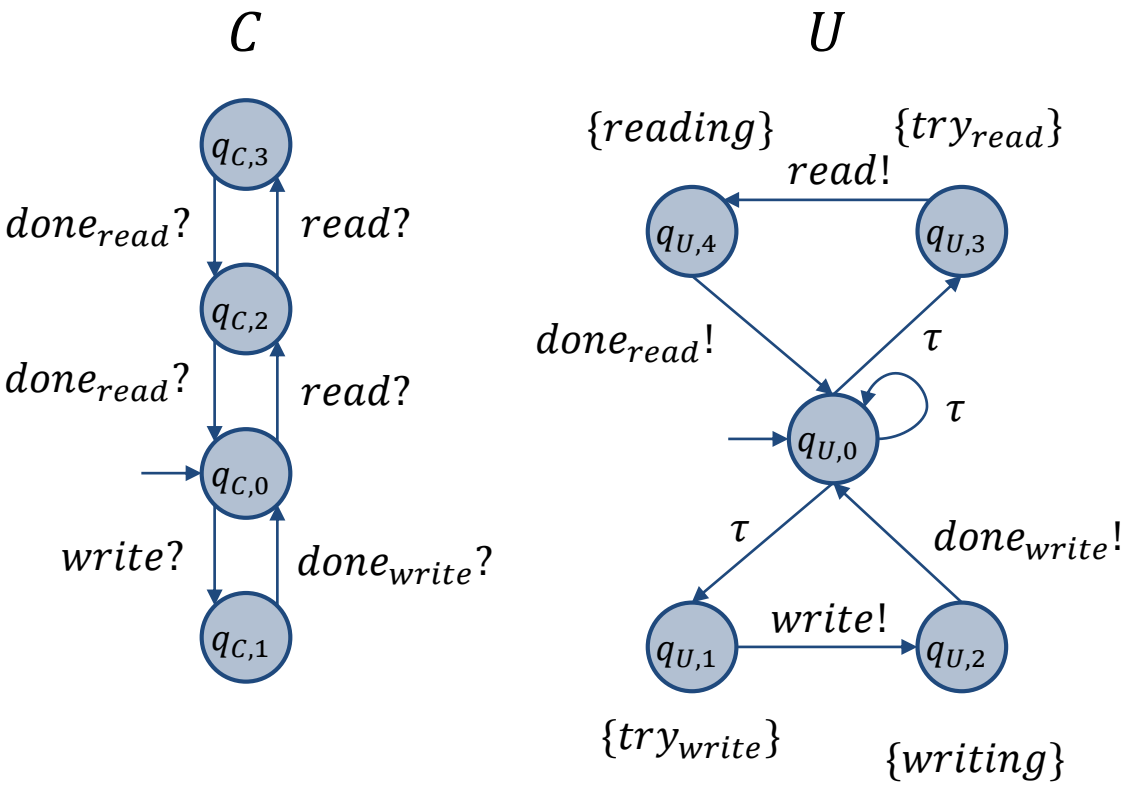
(or, alternatively: the cutoff depends on the size of U, C and A)

Liveness and Fairness

Liveness Properties and Fairness

In concurrent systems (incl. parameterized systems), most liveness properties cannot be satisfied without some notion of fairness.

Example: Consider the system $C \parallel U^2$.
If one of the U-processes moves into $q_{U,1}$, but then never moves again, it will not satisfy our liveness property.



Fairness requires that all processes are allowed to move infinitely often, if they satisfy certain conditions. There are different fairness notions that vary in the conditions they impose.

Different Notions of Fairness

Consider a labeled transition system $(S, S_0, \Sigma_{int} \cup \Sigma_{sync}, \Delta, \Lambda)$, obtained by composition of multiple processes.

In a given state $s \in S$ we say that a process i is enabled if there exists a transition $(s, a, s') \in \Delta$ in which process i participates, i.e., it is either based on an internal transition of process i or a synchronizing transition where process i is sender or receiver.

A run of a system is

- unconditionally fair if every process makes infinitely many local transitions
- strong-fair if every process that is enabled infinitely often makes infinitely many local transitions
- weak-fair if every process that is enabled always after some point will eventually make a local transition

Example: Liveness Properties and Fairness

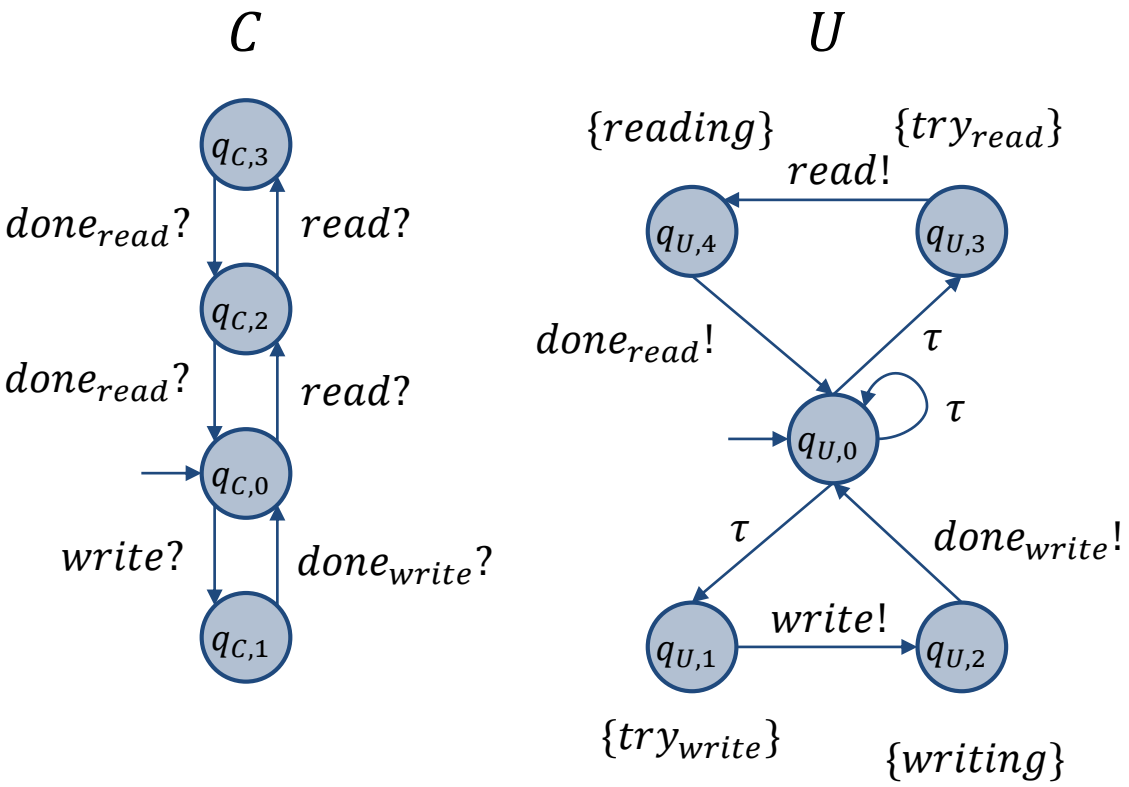
Consider the system $C \parallel U^2$ under different fairness notions.

Suppose one of the U-processes moves into $q_{U,1}$.

Under unconditional fairness, it must eventually be allowed to move into $q_{U,2}$.

Under strong fairness, note that it will be enabled infinitely often, and thus always has to be allowed to move.

Under weak fairness, it may not be allowed to move, since it is not enabled when the controller is in a state different from $q_{C,0}$.



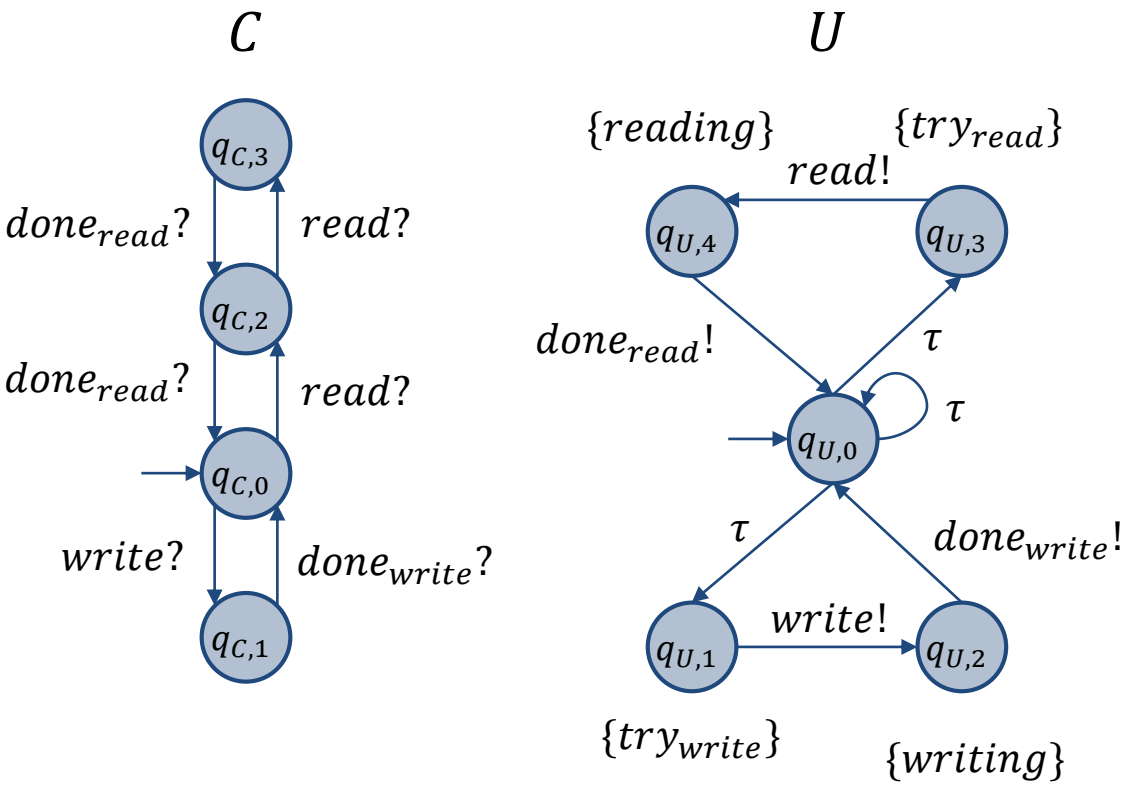
Example: Liveness Properties and Fairness (II)

Now consider the system $C \parallel U^3$ under different fairness notions.

Suppose one of the U-processes moves into $q_{U,1}$.

Under unconditional fairness, it must eventually be allowed to move into $q_{U,2}$.

Under strong fairness, it may happen that (after some point in time) always one of the other U-processes is in $q_{U,4}$, and therefore it will not be infinitely often enabled.



Checking Liveness Properties of PR Systems under Fairness

The PMCP under a notion of fairness asks whether all runs of a parameterized system that satisfy the fairness notion also satisfy the given property.

Theorem: The PMCP under strong fairness for PR systems with finite-state processes and properties defined as Büchi automata is decidable.

Review/Lessons Learned Today

- Example: Checking a Complex Safety Property of the MESI Protocol
- Systems with Pairwise Rendezvous (PR) Synchronization (not as powerful as broadcast)
- Vector Addition Systems (system model that can be seen as modified counter representation)
- Checking Liveness Properties of PR Systems (based on Büchi automata and bounds/cutoffs)
- Liveness and Fairness (can't have one without the other)

Next time: why does the approach for checking liveness not work for broadcast protocols?